type_conversion Â¶

Type conversion helper function between Java and Python types.

JDouble module-attribute Â¶

```
JDouble = autoclass('java.lang.Double')
```

JLong module-attribute Â¶

```
JLong = autoclass('java.lang.Long')
```

JString module-attribute Â¶

```
JString = autoclass('java.lang.String')
```

default_transformation module-attribute Â¶

```
default_transformation: Optional[Callable] = None
```

list_class module-attribute Â¶

```
list_class = find_javaclass('java.util.List')
```

logger module-attribute Â¶

```
logger = getLogger(__name__)
```

map_class module-attribute Â¶

```
map_class = find_javaclass('java.util.Map')
```

option_class module-attribute Â¶

```
option_class = find_javaclass(_option_javaref)
```

optional_class module-attribute Â¶

```
optional_class = find_javaclass('java.util.Optional')
```

pair_class module-attribute Â¶

```
pair_class = find_javaclass(_pair_javaref)
```

set_class module-attribute Â¶

```
set_class = find_javaclass('java.util.Set')
```

transformations `module-attribute` Â¶

```
transformations = {list_class: _transform_list, set_class: _transform_set, map_class: _transform_map, optional_class: _transform_optional, option_class: _transform_optional, pair_class: _transform_feedzai_pair}
```

cast_java_subclass Â¶

```
cast_java_subclass(java_object)
```

Helper method that casts a java class to its real subclass implementation.

For context, for example, imagine the class Format that has as subclasses DateTimeFormat and TimestampFormat.

When java returns an object with type Format and we know it is a TimestampFormat, we first need to cast it to this subclass so we can for example access its getUnit() specific method.

get_optional_or_fail Â¶

```
get_optional_or_fail(optional, fail_message, mapper=None) -> Any
```

Fetches the value stored inside a com.feedzai.commons.Option.

Otherwise, fails and logs an error message with 'fail_message'

Parameters:

Name	Type	Description	Default
`optional` Â¶		The optional from which the value will be extracted.	*required*
`fail_message` Â¶		Message to be logged in the optional is empty.	*required*
`mapper` Â¶		Mapper function.	`None`

Returns:

Type	Description
`Any`	The value stored inside the optional. Fails if the optional is empty.

get_optional_or_none Â¶

```
get_optional_or_none(java_optional, mapper=None)
```

optional_char_or_else Â¶

```
optional_char_or_else(value, or_else)
```

Workaround utility method for com.feedzai.commons.Option type parameters.

Parameters:

Name	Type	Description	Default
`value` Â¶		Option	*required*
`or_else` Â¶		Default value, if the option is empty	*required*

Returns:

Type	Description
`Any`	Option value or 'or_else' if the option is empty

to_feedzai_option Â¶

```
to_feedzai_option(obj: Any)
```

Creates an Option.of(obj).

to_java_list Â¶

```
to_java_list(pylist: Iterable)
```

Create a java list from a python iterable.

to_java_map Â¶

```
to_java_map(py_dict: Mapping)
```

Creates a java HashMap from a python dict.

Parameters:

Name	Type	Description	Default
`py_dict` Â¶	`Mapping`	the dictionary to be converted into an HashMap	*required*

Returns:

Type	Description
`HashMap`	A Java HashMap

to_java_set Â¶

```
to_java_set(pylist: Iterable)
```

Create a java set from a python iterable.

to_python_list Â¶

```
to_python_list(java_list, mapper=None) -> List
```

Create a python list from a java list.

to_python_map Â¶

```
to_python_map(java_map, key_mapper=None, value_mapper=None) -> Dict
```

Create a python map from a java map.

to_python_set Â¶

```
to_python_set(java_set, mapper=None) -> Set
```

Create a python set from a java set.

to_serializable Â¶

```
to_serializable(obj: Any) -> Any
```

Convert int, float, str to their serializable Java version:

- `int` -> `java.lang.Long`
- `float` -> `java.lang.Double`
- `str` -> `java.lang.String`
- None -> None

transform_java_object Â¶

```
transform_java_object(java_object)
```

Transforms a java object into the correspondent python object.

1. If the provided object is already a python object, then it will simply be returned without any change. A java object is defined as an object that implements jnius.JavaClass.
2. If the provided object matches any of the specific transformations in transformations, then the correspondent custom transformation is applied.
3. If none of the above rules apply, then the object will be wrapped in a JavaWrapper object.

Parameters:

Name	Type	Description	Default
`java_object` Â¶		The java object to be transformed.	*required*

Returns:

Type	Description
`Any`	The correspondent python object.